

PDS TECH

ARQUITETURA E DESENVOLVIMENTO EM JAVA

FASE 3 | AULA 04 -

# MONITORAMENTO E ESCALABILIDADE

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

**SUMÁRIO**

O QUE VEM POR AÍ? ..... 3

HANDS ON ..... 4

SAIBA MAIS ..... 5

O QUE VOCÊ VIU NESTA AULA? ..... 16

REFERÊNCIAS ..... 17

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

## O QUE VEM POR AÍ?

Imagine o seguinte cenário: você desenvolve um sistema, coloca em produção e, por se tratar de um sistema crítico, opta por uma arquitetura distribuída, como a de microsserviços. Tudo parece estar indo bem, mas, de repente, começam a surgir reclamações de clientes sobre problemas no funcionamento. O que fazer agora? Como identificar e resolver essas falhas em um ambiente complexo e distribuído?

É exatamente por isso que o tema desta aula é tão importante. Vamos falar sobre ferramentas de monitoramento e como aplicar a escalabilidade, explorando como essas ferramentas podem ser usadas para antecipar problemas, identificar gargalos e otimizar o desempenho, garantindo que sua aplicação esteja sempre pronta para atender às demandas dos seus usuários. Mais do que solucionar problemas, você aprenderá a utilizá-las estrategicamente para prevenir falhas antes que elas impactem seus clientes.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

## HANDS ON

Nesta videoaula, você terá a oportunidade de colocar em prática os conceitos apresentados sobre Monitoramento e Escalabilidade. Os vídeos a seguir detalham como utilizar ferramentas essenciais, como Prometheus, Grafana, ELK Stack e Fluentd, além de estratégias avançadas de escalabilidade em ambientes de nuvem e Kubernetes.

As videoaulas guiam você em um passo a passo sobre o uso de monitoramento de métricas, centralização de logs e ajuste dinâmico de recursos.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

## SAIBA MAIS

Antes de explorarmos o tema de monitoramento e escalabilidade, é importante revisitar os conceitos abordados nas aulas anteriores para termos uma base que facilitará a compreensão dos tópicos atuais.

Nas aulas anteriores, exploramos os fundamentos da Arquitetura Distribuída e as estratégias para garantir a resiliência dos sistemas, culminando na implementação prática desses padrões em Java.

Monitoramento e escalabilidade são pilares essenciais para o funcionamento de sistemas modernos. À medida que a tecnologia avança, os sistemas precisam lidar com volumes cada vez maiores de dados, usuários e interações. Nesta aula, vamos explorar esses conceitos, sua importância e como aplicá-los de forma prática.

**O que é monitoramento?**

O monitoramento é o processo de observar e medir continuamente o desempenho, a saúde e a segurança de um sistema. Ele serve como um “termômetro” que permite identificar gargalos, prever falhas e garantir a estabilidade.

Exemplo prático: imagine um e-commerce em dia de promoção. Se não houver monitoramento, um aumento inesperado de acessos pode causar lentidão ou até mesmo a queda do sistema. Com ferramentas adequadas, é possível monitorar métricas como CPU e tráfego de rede em tempo real, prevenindo problemas.

Nós podemos utilizar as principais métricas no monitoramento, como CPU e memória, que indicam o uso dos recursos do servidor, disco e rede, que avaliam a capacidade de armazenamento, e tráfego de dados. Isso permite analisar a experiência do usuário, observando o tempo de resposta e disponibilidade.

**Dica de leitura: Site Reliability Engineering: How Google Runs Production Systems.**

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

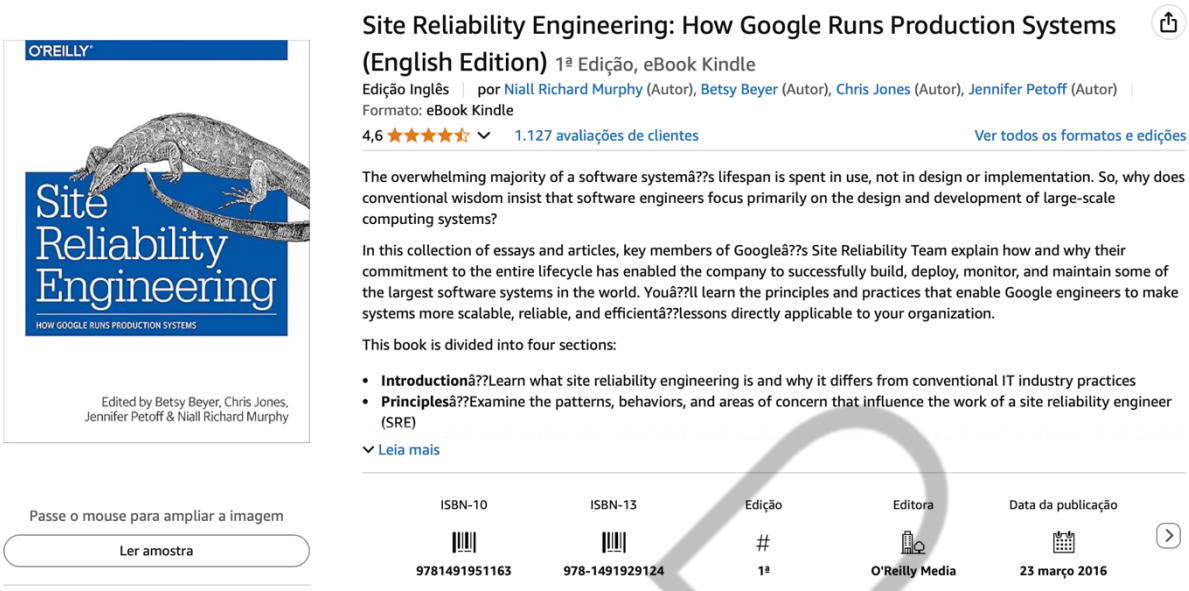



Figura 1 – Livro Site Reliability Engineering
Fonte: elaborado pelo autor (2024)

Se você quer entender como empresas líderes como o Google garantem alta disponibilidade e confiabilidade em seus sistemas, o livro acima é essencial. Ele combina práticas de engenharia de software com operações para criar sistemas robustos, escaláveis e resilientes, oferecendo insights valiosos para qualquer profissional de tecnologia que busque excelência operacional.

Você já se perguntou como seria um dashboard de monitoramento eficaz? Imagine uma interface que não apenas exibe métricas e indicadores, mas que oferece uma visão clara e instantânea da saúde do sistema. Um bom dashboard vai além de gráficos e números: ele fornece insights acionáveis, permitindo que equipes de tecnologia identifiquem problemas rapidamente e tomem decisões informadas.

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com




Figura 2 – Grafana
Fonte: barre.arq.br (2024)

Monitorar um sistema distribuído vai muito além de saber se ele está “no ar” ou não. Trata-se de observar o comportamento de todos os componentes de um sistema: servidores, serviços, APIs, bancos de dados, filas de mensagens, entre outros. Em arquiteturas distribuídas, onde diferentes partes do sistema podem estar localizadas em servidores e regiões distintas, o monitoramento se torna essencial para garantir a saúde e o desempenho geral.

Um bom monitoramento deve responder perguntas como:

- Qual o desempenho de cada serviço?
- Existem gargalos em algum ponto da arquitetura?
- Como estão as métricas de uso, como CPU, memória e rede?
- Como essas informações se traduzem na experiência do usuário?

Explorando um pouco mais sobre monitoramento, precisamos falar sobre as boas práticas, como definir métricas relevantes. Não monitore tudo de forma indiscriminada. Escolha métricas que tragam valor, como latência de APIs, taxa de erros, tempo médio entre falhas (MTBF) e tempo médio de recuperação (MTTR). Lembre-se de trabalhar com a métrica de percentil.

Implementar log estruturados e ricos em informações permite rastrear o comportamento de um sistema. Use formatos como JSON para facilitar a análise e a correlação entre eventos.

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

Centralizar dados com ferramentas de agregação de logs e métricas, como Prometheus, ELK Stack ou Datadog, permite visualizar o sistema como um todo em vez de fragmentá-lo em partes isoladas.

Lembre-se de criar alertas inteligentes, configurando para que sejam acionados apenas quando algo realmente relevante acontecer. Um sistema de alertas muito sensível pode causar “falso positivo,” levando equipes a ignorarem sinais críticos.

Monitoramento em tempo real e histórico é essencial. Combine dashboards em tempo real com dados históricos para identificar tendências, prever problemas e otimizar os recursos.

O Grafana é uma das ferramentas mais populares para visualizar dados de monitoramento. Ele permite criar dashboards interativos e dinâmicos, que centralizam métricas de diferentes fontes como Prometheus, InfluxDB e ElasticSearch.

Imagine que estamos monitorando uma API em uma arquitetura distribuída. No Grafana, poderíamos configurar um dashboard com os seguintes painéis:

**Latência das requisições:** mostrando o tempo médio de resposta das APIs.

**Taxa de erros (HTTP 5xx):** um gráfico indicando a quantidade de erros em diferentes serviços.

**Uso de recursos:** um painel para CPU, memória e utilização de disco de cada servidor.

**Exemplo prático com o Grafana**

Pense no seguinte cenário: temos uma API hospedada em um cluster Kubernetes e estamos coletando métricas com o Prometheus. Podemos criar um painel no Grafana com uma query para medir a latência média.

```
histogram_quantile(0.95,sum(rate(http_request_duration_seconds_bucket[5m])) by (le))
```

Código-fonte 1 – Exemplo de query para Prometheus
Fonte: elaborado pelo autor (2024)

Essa métrica mostrará a latência no percentil 95, permitindo entender o tempo de resposta da maior parte das requisições. Além disso, alertas podem ser configurados diretamente no Grafana para notificar a equipe caso a latência ultrapasse um limite aceitável, como 500ms.

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

O monitoramento eficaz é um dos pilares de uma arquitetura de sistemas distribuídos bem-sucedida. Ele permite antecipar problemas, otimizar a experiência do usuário e, principalmente, garantir a continuidade do serviço.

Com boas práticas e ferramentas, como o Grafana, transformar dados em ações concretas se torna muito mais acessível. Adotar uma abordagem estruturada para monitorar e interpretar métricas é o primeiro passo para elevar a resiliência e a confiabilidade do seu sistema distribuído.

À medida que nós exploramos o monitoramento e seu papel em garantir a estabilidade dos sistemas distribuídos, surge uma pergunta inevitável: como esses sistemas podem crescer de forma eficiente para atender à demanda crescente de usuários e transações?

É aqui que entra o conceito de escalabilidade. Ela é um dos principais objetivos ao projetar arquiteturas modernas, especialmente em ambientes distribuídos, onde a demanda pode variar drasticamente em questão de segundos.

**Mas afinal o que é escalabilidade?**

Escalabilidade refere-se à capacidade de um sistema de crescer ou se adaptar às mudanças na demanda sem perder desempenho. É um dos temas mais relevantes em nossa jornada pelo universo da arquitetura de sistemas.

Mas afinal, o que significa escalabilidade? Por que ela é tão importante em sistemas modernos e, principalmente, como podemos garantir que nossos sistemas sejam escaláveis para atender ao crescimento da demanda sem comprometer o desempenho? Vamos oferecer insights teóricos e práticos para que você entenda e aplique os conceitos de escalabilidade no desenvolvimento de suas aplicações.

Escalabilidade é a capacidade de um sistema de lidar eficientemente com um aumento ou diminuição na carga de trabalho sem sacrificar o desempenho, estabilidade ou a experiência do usuário. Em termos práticos, significa que, mesmo que o número de usuários ou requisições aumente drasticamente, o sistema continuará operando com a mesma velocidade, consistência e confiabilidade.

Um exemplo clássico de escalabilidade está no tempo de resposta de uma aplicação. Imagine que, em condições normais, uma API leva 200ms para responder a uma solicitação. Em um sistema escalável, mesmo com milhares de requisições

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

simultâneas, esse tempo deve permanecer constante, ou pelo menos dentro de um intervalo aceitável. Caso contrário, o sistema poderá enfrentar gargalos, falhas e, eventualmente, a insatisfação dos usuários.

### Tipos de Escalabilidade




Figura 2 – Vertical vs Horizontal
Fonte: bkappi.com.br (2024)

A escala vertical, também chamada de scaling up, envolve aumentar os recursos de um único servidor, como adicionar mais memória RAM, aumentar a capacidade do processador ou trocar para um hardware mais potente. Essa abordagem é relativamente simples e não requer alterações no código ou na arquitetura da aplicação.

Por exemplo, imagine um servidor com 8GB de RAM que começa a apresentar lentidão devido ao aumento da carga. Escalar verticalmente significa trocar esse servidor por um com 16GB ou mais. Essa solução, no entanto, tem limitações: há um teto para o quanto um único servidor pode ser melhorado e o custo pode se tornar proibitivo.

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

A escala horizontal, ou scaling out, envolve adicionar mais servidores ao ambiente e distribuir a carga entre eles. Essa abordagem é comum em sistemas distribuídos, onde várias máquinas trabalham em conjunto para lidar com a carga de trabalho.

Por exemplo, ao invés de ter um único servidor potente, você pode configurar um cluster com vários servidores menores, balanceando a carga entre eles. Essa abordagem é mais escalável a longo prazo, mas exige mais planejamento e implementação cuidadosa, incluindo balanceamento de carga, orquestração e arquiteturas sem estado (stateless).

Uma métrica essencial para avaliar a escalabilidade é a latência, que mede o tempo que um sistema leva para responder a uma solicitação. Em um sistema bem escalado, a latência deve permanecer constante, mesmo com aumento na carga.

Existem dois tipos principais de latência a serem considerados:

**Latência de rede:** refere-se ao tempo que os dados levam para viajar entre o cliente e o servidor. Para reduzir essa latência, é comum o uso de CDNs (Content Delivery Networks), que armazenam dados em servidores distribuídos geograficamente próximos aos usuários.

**Latência do aplicativo:** é o tempo que a aplicação leva para processar a solicitação após recebê-la. Essa latência pode ser reduzida com otimizações no código, uso de cache, consultas eficientes ao banco de dados e balanceamento de carga.

Manter a latência baixa, mesmo sob alta carga, é um indicador claro de que o sistema está bem escalado.

**Como monitoramento e escalabilidade se conectam?**

A relação entre monitoramento e escalabilidade é fundamental para a construção de sistemas modernos e resilientes. Um sistema bem monitorado é capaz de identificar padrões de uso, antecipar problemas e aplicar escalabilidade no momento certo.

**Ferramentas essenciais de monitoramento**

Para implementar um monitoramento eficaz, diversas ferramentas populares são amplamente utilizadas, cada uma atendendo a necessidades específicas:

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

### Prometheus

O Prometheus é ideal para coleta de métricas em sistemas distribuídos. Ele utiliza "exporters" para capturar dados de diferentes fontes, como bancos de dados, microsserviços e filas de mensagens. Sua capacidade de lidar com ambientes complexos e sua integração com outras ferramentas, como o Grafana, fazem dele uma escolha robusta para arquiteturas modernas.

### Grafana

O Grafana destaca-se na visualização de métricas, permitindo a criação de dashboards dinâmicos e interativos para análise em tempo real. Além disso, seus recursos avançados de alerta ajudam equipes a responder rapidamente a anomalias, reduzindo o impacto de possíveis problemas no sistema.

A figura a seguir mostra uma arquitetura de sistemas utilizando o Grafana e Prometheus com seus sistemas.

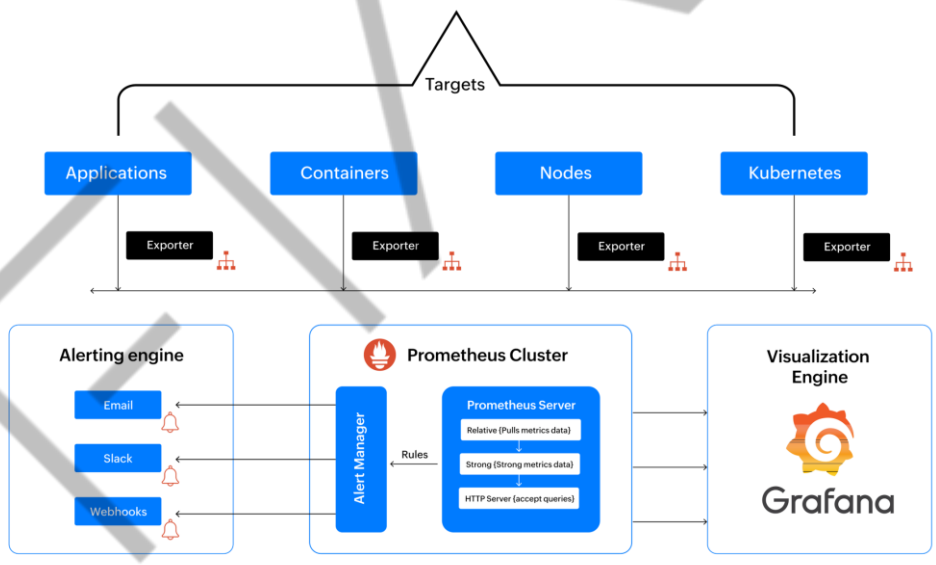



Figura 3 – Monitoring stack architecture with Prometheus and Grafana
Fonte: site24x7.com (2024)

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

**Zabbix**

Focado no monitoramento de infraestrutura, o Zabbix rastreia recursos físicos e virtuais, como servidores, máquinas virtuais e dispositivos IoT. Ele assegura que a base da aplicação, incluindo hardware e redes, opere dentro dos padrões esperados.

Essas ferramentas desempenham papéis complementares, garantindo visibilidade tanto na camada de infraestrutura quanto na camada de aplicação.

**Application Performance Monitoring (APM)**

Além das ferramentas tradicionais, o APM (Application Performance Monitoring) se destaca como uma solução essencial para monitorar e otimizar o desempenho das aplicações. Ele foca em métricas específicas do comportamento das aplicações, permitindo identificar gargalos em tempo real e melhorar a experiência do usuário. Com o APM, é possível monitorar:

- **Tempo de resposta** de APIs e microsserviços, identificando os serviços que impactam a performance.
- **Taxas de erro e exceções**, correlacionando eventos para identificar causas-raiz.
- **Transações completas**, analisando fluxos operacionais e localizando pontos de lentidão.
- **Comportamento em tempo real**, possibilitando ajustes imediatos para evitar impacto nos usuários.

O APM é particularmente útil em arquiteturas baseadas em microsserviços, onde cada componente tem um papel crítico no funcionamento do sistema.

**Arquitetura combinando Grafana, Prometheus e APM**

Uma arquitetura típica combinando essas ferramentas pode ser configurada da seguinte forma:

1. **Coleta de métricas**: o Prometheus utiliza "exporters" para capturar dados, como latência de requisições, consumo de CPU e memória, e taxas de erro nos microsserviços.
2. **Visualização em tempo real**: essas métricas são enviadas ao Grafana, que transforma os dados em dashboards. Por exemplo:

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

- Gráficos de latência e erros.
- Visualizações de utilização de recursos.
- Alertas para thresholds críticos.

3. **Análise detalhada com APM**: o APM complementa essa visão, detalhando:
   - Fluxos de transações completas, ajudando a identificar componentes problemáticos.
   - Operações mais lentas, como consultas ao banco de dados ou APIs externas.
   - Dados históricos para análise de tendências e planejamento de capacidade.

Outro ponto importante no monitoramento é a centralização de logs. Os logs separados dificultam a análise e aumentam o tempo de resposta em caso de falhas. Ferramentas como o ELK Stack (ElasticSearch, Logstash e Kibana) permitem consolidar e analisar logs de diferentes partes do sistema em um único painel. A ELK Stack é um conjunto de ferramentas desenvolvido pela Elastic, que combina três componentes principais para criar uma solução robusta de gerenciamento de logs com ElasticSearch.

ElasticSearch é um mecanismo de busca e análise baseado em documentos que armazena logs de maneira estruturada. Ele utiliza um formato de índice que permite consultas rápidas e detalhadas.

Logstash é uma ferramenta que coleta, processa e transforma logs antes de enviá-los para o ElasticSearch. Suas funcionalidades principais são filtragem e formatação de logs de diferentes fontes, como arquivos de texto, bancos de dados ou APIs. O uso prático é transformar logs brutos em dados estruturados para análise.

Kibana é a interface gráfica do ELK Stack, usada para criar dashboards interativos e gráficos baseados nos logs armazenados no ElasticSearch, e tem como funcionalidades principais a criação de visualizações, como gráficos de linha, barras e mapas, e a configuração de alertas para eventos específicos (por exemplo, um aumento no número de erros HTTP 500).

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

Alternativamente ao ELK Stack, o Fluentd é uma opção leve e modular para integrar sistemas diversos e centralizar informações em tempo real, focada em integrar diferentes fontes de dados e facilitar a centralização de logs. Foi desenvolvido pela Treasure Data e é amplamente utilizado em ambientes de conteinerização, como Kubernetes.

Fluentd pode coletar logs de diversas fontes, como arquivos de log locais, APIs de serviços externos ou fluxos de eventos de aplicações. Com essas ferramentas, é possível detectar padrões, rastrear erros e realizar auditorias de maneira eficiente.

**Quando usar cada uma?**

**ELK Stack**

Se você precisa de uma solução robusta, capaz de armazenar grandes volumes de dados e oferecer análise avançada, o ELK Stack é a escolha ideal. Ele é especialmente útil para equipes que desejam ter controle total sobre logs e criar dashboards interativos.

**Fluentd**

Para ambientes menores ou distribuídos, onde a simplicidade e a eficiência são prioritárias, o Fluentd é mais adequado. Ele também é uma excelente escolha para sistemas que precisam enviar logs para múltiplos destinos.

Ambas as ferramentas desempenham um papel vital na centralização de logs e podem ser utilizadas de forma complementar, dependendo das necessidades do sistema.

Chegamos ao final da nossa aula, onde exploramos conceitos fundamentais, boas práticas e ferramentas essenciais para garantir que sistemas distribuídos sejam eficientes, escaláveis e resilientes. Nesta jornada, discutimos como o monitoramento permite identificar gargalos e prever problemas, enquanto a escalabilidade assegura que os sistemas acompanhem o crescimento da demanda.

Para entender com mais profundidade como cada ferramenta funciona e aprender a utilizá-las na prática, recomendo que vocês assistam aos vídeos disponibilizados nesta aula.

**PDF exclusivo para Michelle Ayumi Kashima - rm371026**
**michelleayumi14@gmail.com**

## O QUE VOCÊ VIU NESTA AULA?

Nesta aula, exploramos os fundamentos e a prática de Monitoramento e Escalabilidade em sistemas. Você aprendeu a importância do monitoramento para identificar gargalos e prever falhas, como centralizar logs e métricas utilizando ferramentas como ELK Stack e Fluentd.

Vimos também boas práticas para configurar alertas eficientes e se concentrar nas métricas mais relevantes para o negócio e como integrar monitoramento e escalabilidade para criar sistemas robustos e resilientes.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

# REFERÊNCIAS

BARRE, A. **Explorando o Grafana: monitoramento e visualização de dados simplificados.** Disponível em: https://barre.arq.br/site/explorando-o-grafana-monitoramento-e-visualizacao-de-dados-simplificados/ . Acesso em: 13 dez. 2024.

BETT, B.; CHALMERS, J.; SLOAN, C. **Site Reliability Engineering: How Google Runs Production Systems.** [S.l.]: O'Reilly Media, 2016.

BKAPPI. **Escalabilidade Vertical e Horizontal em Sistemas**. Disponível em: https://bkappi.com/blog/escalabilidade-vertical-horizontal-sistemas/. Acesso em: 13 dez. 2024.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

## PALAVRAS-CHAVE

Performance. Monitoramento. Escalabilidade.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com




PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com